

БУ ВО
«СУРГУТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ХАНТЫ-МАНСКИЙСКОГО АВТОНОМНОГО ОКРУГА – ЮГРЫ»
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ

Кафедра информатики и вычислительной техники

Курсовая работа
по дисциплине «Разработка web-приложений»
на тему «чат поддержки, база знаний и поиск»

Выполнил: студент группы 607-42

Говор Сергей Владимирович

Проверил: к. т. н., доцент кафедры ИВТ

Егоров Александр Алексеевич

Сургут – 2026

СОДЕРЖАНИЕ

Введение.....	3
1. МОДЕЛЬ РАЗВЕРТЫВАНИЯ СИСТЕМЫ.....	4
1.1. Диаграмма развертывания UML.....	4
1.2. Описание развертывания на виртуальном сервере.....	5
2. ПРОЕКТИРОВАНИЕ КОМПОНЕНТА.....	8
2.1. Назначение и границы компонента.....	8
2.2. Модель данных и диаграмма классов.....	9
2.3. Диаграмма компонентов и API-контракты.....	11
2.4. Бизнес-правила и ограничения доступа.....	13
3. UI КОМПОНЕНТЫ И РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ.....	16
3.1. Перечень экранов и состояний.....	16
3.2. Формы, фильтры и валидация.....	17
3.3. Пользовательские сценарии.....	17
3.4. Скриншоты компонента.....	19
Заключение.....	22
Список используемых источников.....	23
Приложение а. plantuml-код диаграмм.....	24

Введение

Настоящая курсовая работа посвящена проектированию и разработке сквозного компонента «Чат поддержки, база знаний и поиск». Данный компонент переносит процесс коммуникации пользователей и внутренней службы технической поддержки в единый контур веб-приложения Notary (интерфейс пользователей — проектируется).

Со стороны заявителя (интерфейс — проектируется) компонент решает задачи быстрого поиска ответов на часто задаваемые вопросы (FAQ) в базе знаний (проектируется) посредством полнотекстового поиска по статьям, а при необходимости — создания официального тикета (обращения) в службу поддержки с последующим ведением текстового диалога.

Со стороны оператора внутренней поддержки компонент предоставляет автоматизированное рабочее место — очередь тикетов, распределение обращений по уровням приоритета, ведение диалога с заявителем, а также инструменты создания и редактирования справочных статей для расширения базы знаний (проектируется).

Цель работы — разработать, спроектировать и задокументировать компонент чата поддержки, базы знаний и поиска для веб-портала Notary, включая пользовательские интерфейсы заявителя (проектируется) и оператора, серверную бизнес-логику управления жизненным циклом обращений, подсистему контроля SLA и модуль оповещений (проектируется).

Для достижения поставленной цели требуется решить следующие задачи:

1. Изучить индивидуальное задание, архитектурные шаблоны проекта и определить границы ответственности компонента.
2. Построить диаграмму развертывания UML проекта и детально описать инфраструктурную схему функционирования подсистемы поддержки.
3. Разработать реляционную модель данных, построить диаграмму классов UML и связать сущности поддержки с общими сущностями пользователей системы.
4. Спроектировать диаграмму компонентов UML с разделением на подсистемы чата и базы знаний, составить спецификацию API-контрактов взаимодействия.
5. Сформулировать строгие бизнес-правила обработки тикетов и разграничения ролевого доступа.
6. Разработать и описать интерфейсные решения (Angular-компоненты), формы ввода, логику валидации, фильтрации и составить подробное руководство пользователя с эксплуатационными сценариями.

Объект разработки — веб-система Notary, предназначенная для автоматизации нотариальной оценки и взаимодействия с гражданами.

Предмет разработки — изолированный компонент поддержки, чата, ведения базы знаний и поиска (проектируется) со встроенным механизмом контроля SLA и уведомлений (проектируется).

1. МОДЕЛЬ РАЗВЕРТЫВАНИЯ СИСТЕМЫ

1.1. Диаграмма развертывания UML

Проект разворачивается на виртуальном сервере под управлением операционной системы семейства Linux с использованием технологии контейнеризации Docker и оркестрации Docker Compose. Взаимодействие пользователя с системой осуществляется через современный веб-браузер, в котором выполняется клиентское одностраничное приложение (Angular SPA). Раздачу статических файлов клиентского приложения, а также функции обратного прокси-сервера (reverse proxy) выполняет контейнер nginx portal. Он принимает входящий HTTP-трафик на порту 80 (HTTPS при необходимости обеспечивает Nginx Proxy Manager) и перенаправляет вызовы RPC-процедур Connect (маршруты /notary.support.*) и REST API (маршруты /api/*) на backend-контейнер NestJS API. Данный подход позволяет клиенту и серверу функционировать в рамках одного источника (Same-Origin policy).

Сервер приложений NestJS осуществляет обработку бизнес-логики, проверку ролей, вычисление SLA и взаимодействует со СУБД PostgreSQL через объектно-реляционное отображение Prisma ORM.

Ниже представлена диаграмма развертывания, на которой акцентировано внимание на контуре прохождения данных компонента поддержки и справки.

Диаграмма развертывания системы: Модуль поддержки и справки

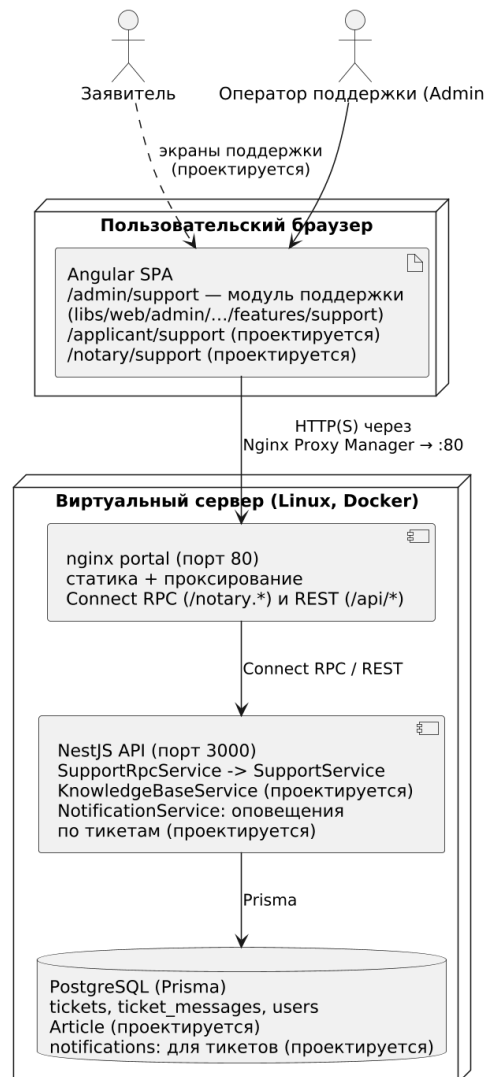


Рисунок 1 – Диаграмма развертывания

1.2. Описание развертывания на виртуальном сервере

Вся базовая инфраструктура проекта описывается в декларативных файлах `docker-compose.yaml` и `apps/web/docker-compose.portal.yml`. База данных PostgreSQL выступает единым реляционным хранилищем для всех системных модулей. В контексте разработки компонента поддержки, в базу внедрены таблицы для хранения структуры тикетов, истории сообщений в чате, текстового содержимого статей базы знаний (проектируется) и лога системных уведомлений (для тикетов — проектируется).

Сборка образов приложений автоматизирована с помощью многоэтапных Docker-файлов (Dockerfile). Для клиентской части на первом этапе (Build Stage) среда Node.js компилирует production-сборку Angular SPA, оптимизируя исходный TypeScript-код, а на втором этапе полученные артефакты переносятся в легковесный образ Nginx, обеспечивающий высокую скорость отдачи статического контента.

Безопасность и изоляция данных на уровне интерфейса гарантируются встроенными механизмами Angular-роутинга (Route Guards), которые проверяют авторизационный токен пользователя и его принадлежность к требуемой роли. Заявителям полностью заблокирован доступ к административной панели и очереди тикетов операторов, а создавать тикеты от чужого имени может только оператор (Admin).

Все критически важные параметры конфигурации вынесены из исходного кода в переменные окружения, считываемые сервером на этапе инициализации:

- DATABASE_URL — строка подключения к PostgreSQL для Prisma.
- JWT_ACCESS_SECRET — криптографический ключ для верификации подписей токенов сессий.

Таблица 1 – Компоненты инфраструктуры и их связь с компонентом поддержки

Элемент	Назначение	Связь с компонентом поддержки
Браузер, Angular SPA	Пользовательский интерфейс портала	Экран /admin/support (только Admin: authGuard, roleGuard); вызовы SupportService по Connect RPC
nginx portal	Контейнер portal: отдаёт статику SPA, проксирует /api/* и Connect RPC (/notary.*) на api:3000; порт 80	Передаёт вызовы SupportService в API; SPA и API работают в одном origin
NestJS API	Контейнер api, порт 3000: бизнес-логика и Connect RPC; проверка JWT в AuthInterceptor	SupportModule: SupportRpcService → SupportService; 9 RPC, права доступа, расчёт SLA
PostgreSQL + Prisma	Реляционное хранилище данных всех модулей	Таблицы tickets, ticket_messages, users (модели Ticket, TicketMessage, User)
Мониторинг: Prometheus, Grafana, Loki, Promtail, Tempo	Метрики, логи и трассировки контейнеров и API	Общие HTTP-логи и метрики API охватывают и вызовы SupportService; отдельных метрик поддержки нет
MinIO: вложения поддержки (проектируется)	S3-хранилище файлов; бакеты payment-documents, assessment-files	Хранение вложений к сообщениям не реализовано: в БД только attachmentIds

Элемент	Назначение	Связь с компонентом поддержки
GigaChat (проектируется)	Внешний сервис ответов ИИ-ассистента	AskSupportAi возвращает шаблонный ответ; внешнего вызова нет

2. ПРОЕКТИРОВАНИЕ КОМПОНЕНТА

2.1. Назначение и границы компонента

Разрабатываемый компонент поддержки и базы знаний является изолированной подсистемой, предназначенной для агрегации пользовательских обращений и предоставления справочной информации (проектируется). Взаимодействие с компонентом строго разграничено в зависимости от роли участника:

1. Заявитель (Applicant):

- Осуществляет поиск по ключевым словам среди опубликованных статей базы знаний (проектируется).
- Просматривает структурированный FAQ по категориям (проектируется).
- Создает новые тикеты с указанием темы, подробного описания проблемы и степени критичности инцидента (экран заявителя — проектируется).
- Ведет диалог в чате с оператором внутри конкретного тикета, отслеживая смену его статусов (экран заявителя — проектируется).

2. Оператор поддержки (роль Admin):

- Работает со специализированным интерфейсом — очередью входящих обращений (Ticket Queue), с сортировкой по дате, приоритету и SLA.
- Берет тикеты в работу, осуществляет консультации в чате и закрывает решенные вопросы.

3. Автор контента / Редактор (роль Admin):

- Осуществляет администрирование базы знаний (проектируется): создает новые статьи, редактирует существующие материалы, распределяет их по тематическим категориям.

Внешние системы (такие как модуль нотариальной оценки недвижимости, подсистема биллинга и генерации копий документов) находятся вне границ проектируемого компонента и взаимодействуют с ним исключительно на уровне общих идентификаторов пользователей (userId).

Таблица 2 – Основные характеристики компонента поддержки

Элемент	Описание
Роль пользователя	Admin — оператор поддержки, экран /admin/support. Applicant и Notary работают со своими тикетами через API; их экраны /applicant/support и /notary/support (проектируется)
Основной сценарий	Оператор выбирает тикет в очереди, нажимает «Взять» (Open → InProgress), отвечает в чате, отмечает «Решено» (Resolved) или закрывает тикет (Closed) с итоговым решением. Вопрос ИИ-ассистенту и эскалация на оператора — есть в API, экран (проектируется)

Элемент	Описание
Входные данные	Форма тикета: тема, email пользователя, приоритет SLA, описание; текст ответа; фильтры очереди (статус, SLA, приоритет) и сортировка; итоговое решение при закрытии. Поисковый запрос по статьям (проектируется)
Выходные данные	Записи в tickets и ticket_messages; срок SLA (slaDeadline); статус тикета; счётчики «Открытых», «Нарушено SLA», «Решено сегодня». Список найденных статей (проектируется)
Соседние модули	Авторизация: AuthInterceptor, requireAuth, requireRole, requireSelfOrRole; пользователи: User, поиск по email при создании тикета; Prisma. Вложения в MinIO (проектируется); уведомления не подключены
Ограничения	Все RPC — только с токеном; не-Admin видит только свои тикеты; менять статус и закрывать тикет может только Admin; в закрытый тикет нельзя писать и менять ему статус; методов правки и удаления сообщений нет

2.2. Модель данных и диаграмма классов

Для обеспечения целостности и нормализации данных спроектирована схема реляционных таблиц. Центральной сущностью подсистемы общения является класс Ticket (обращение), который связан отношением «многие к одному» с сущностью пользователя User (как в роли создателя-клиента authorId, так и в роли исполнителя assigneeId). Каждое обращение содержит лог текстовых сообщений TicketMessage.

Подсистема справки (проектируется) базируется на сущности Article (статья базы знаний), жестко привязанной к её создателю (authorId). При отправке важных ответов или смене ответственных лиц система порождает записи в таблице Notification (проектируется).

Ниже представлена концептуальная диаграмма классов UML, содержащая типы данных, первичные и внешние ключи, а также кратности связей.

Диаграмма классов: чат поддержки, база знаний и поиск

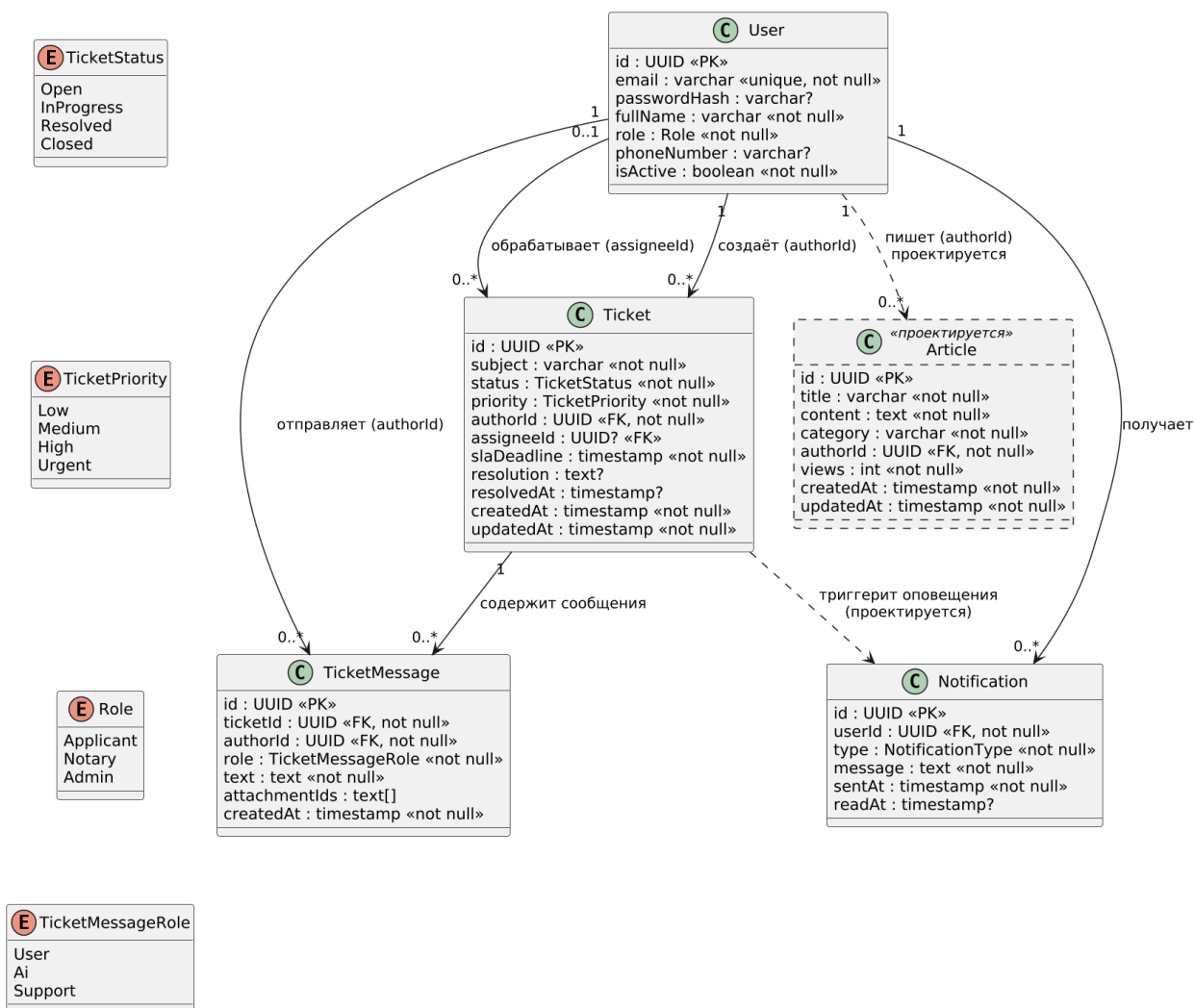


Рисунок 2 – Диаграмма классов

Таблица 3 – Сущности компонента

Сущность	Назначение в компоненте	Ключевые поля и связи
Ticket (tickets)	Обращение в поддержку	id (UUID, PK); subject; status: TicketStatus — Open, InProgress, Resolved, Closed (по умолчанию Open); priority: TicketPriority — Low, Medium, High, Urgent (по умолчанию Medium); authorId → User; assigneeId → User, необязательный; slaDeadline; resolution; resolvedAt; createdAt, updatedAt; messages: TicketMessage[] (1:N)

Сущность	Назначение в компоненте	Ключевые поля и связи
TicketMessage (ticket_messages)	Сообщение в переписке по тикету	id (UUID, PK); ticketId → Ticket, каскадное удаление; authorId → User; role: TicketMessageRole — User, Ai, Support (по умолчанию User); text; attachmentIds: String[] — идентификаторы вложений; createdAt
User (users)	Автор тикетов и сообщений; Admin — исполнитель	id (UUID, PK); email (unique); fullName; role: Role — Applicant, Notary, Admin (по умолчанию Applicant); supportTickets, assignedTickets: Ticket[] (1:N); ticketMessages: TicketMessage[] (1:N)
Article (проектируется)	Статья базы знаний для FAQ и поиска	Концептуальная сущность, в schema.prisma отсутствует; планируемые поля: id, title, content, category, authorId → User

2.3. Диаграмма компонентов и API-контракты

В соответствии с требованиями по четкому разделению ответственности подсистем, архитектура компонента разделена на два крупных домена: Подсистема тикетов и чата (отвечает за оперативную поддержку) и Подсистема базы знаний (проектируется; отвечает за статический контент и поиск).

Взаимодействие между Angular-модулями и RPC-сервисами NestJS специфицировано посредством строгих контрактов протокола Connect RPC (Protobuf).

Диаграмма компонентов: Чат поддержки и База знаний

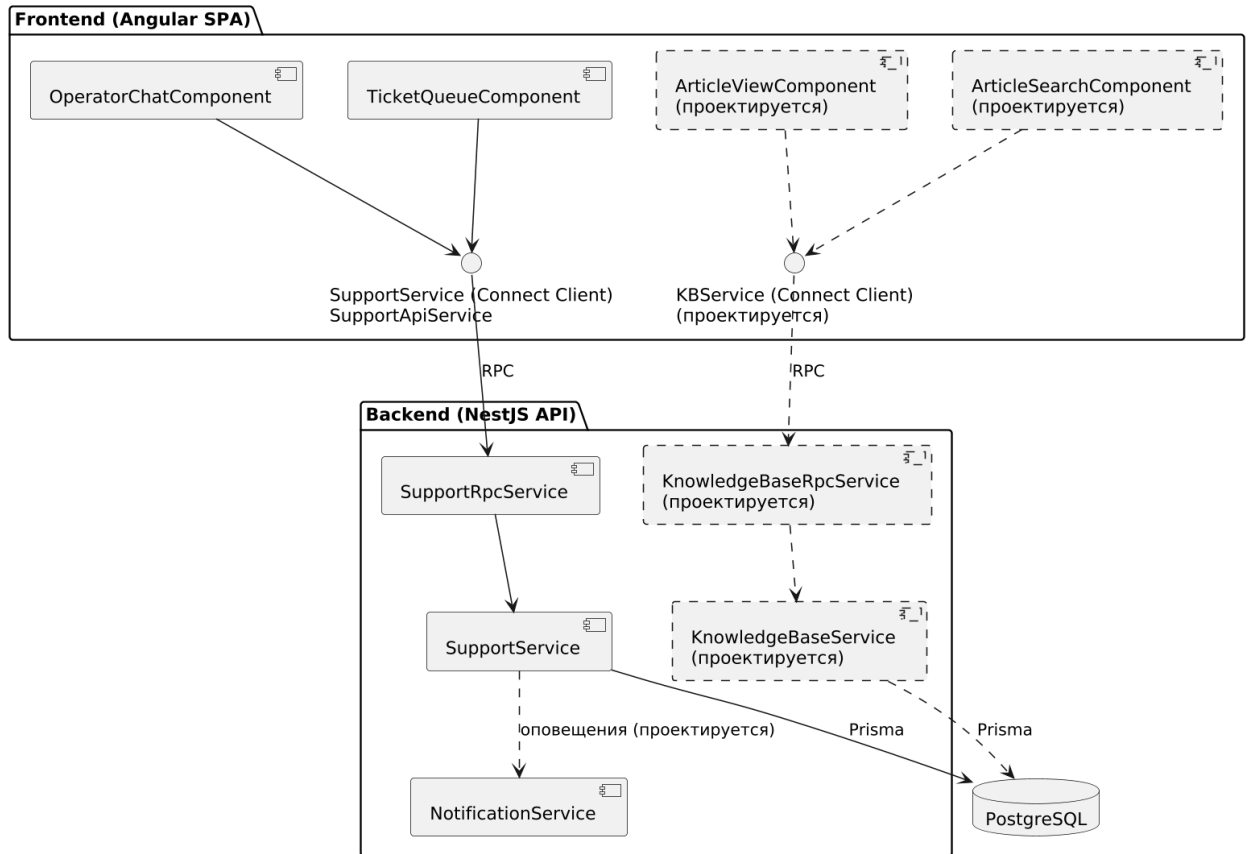


Рисунок 3 – Диаграмма компонентов

Таблица 4 – API-методы компонента

RPC	Назначение	Доступ	Параметры → ответ	Ошибки	Статус
SupportService. AskSupportAi	Ответ ИИ-ассистента на вопрос	Все роли	question_text → answer_text	Unauthenticated; InvalidArgument (пустой вопрос)	Реализован (ответ-заглушка)
SupportService. EscalateToOperator	Тикет из диалога с ИИ: приоритет Medium, история — в сообщении	Все роли	subject, history[] (role, text) → ticket	Unauthenticated; InvalidArgument (пустой subject)	Реализован
SupportService. CreateTicket	Создать тикет; text — первое сообщение	Все роли; от имени другого только Admin	subject, text, priority, author_email (опц.) → ticket	Unauthenticated; InvalidArgument (пустой subject); PermissionDenied; NotFound (нет пользователя с email)	Реализован
SupportService. GetTicket	Карточка тикета	Автор или Admin	id (UUID) → ticket	Unauthenticated; InvalidArgument (не UUID); NotFound; PermissionDenied	Реализован
SupportService. ListTickets	Список тикетов, новые сверху	Admin: все; другие: свои	pagination (page, limit ≤ 200), status_filter → tickets[], meta	Unauthenticated	Реализован

RPC	Назначение	Доступ	Параметры → ответ	Ошибки	Статус
SupportService. AddMessage	Сообщение в тикет; ответ Admin назначает его исполнителем, Open/Resolved → InProgress	Автор или Admin	ticket_id, text, attachment_ids → message, ticket	Unauthenticated; InvalidArgument; NotFound; PermissionDenied; FailedPrecondition (тикет закрыт)	Реализован
SupportService. ListMessages	История сообщений в порядке создания	Автор или Admin	ticket_id, pagination → messages[], meta	Unauthenticated; InvalidArgument; NotFound; PermissionDenied	Реализован
SupportService. UpdateTicketStatus	Сменить статус; InProgress назначает исполнителя, Resolved и Closed ставят resolvedAt	Только Admin	id, status → ticket	Unauthenticated; PermissionDenied; InvalidArgument; NotFound; FailedPrecondition (тикет закрыт)	Реализован
SupportService. CloseTicket	Закрыть тикет с итоговым решением	Только Admin	id, resolution → ticket	Unauthenticated; PermissionDenied; InvalidArgument; NotFound; FailedPrecondition (уже закрыт)	Реализован
KBService. SearchArticles	Поиск статей базы знаний	Все роли	query, category → articles[]	Unauthenticated	Проектируется
KBService. CreateArticle	Создание статьи базы знаний	Только Admin	title, content, category → article	Unauthenticated; PermissionDenied; InvalidArgument	Проектируется

Примечание: контракт —

libs/shared/api-contracts/proto/notary/support/v1alpha1/support.proto;

обработчики — libs/api/support/src/lib/support-rpc.service.ts и support.service.ts;

регистрация — apps/api/src/app/connect-router.registry.ts. Unauthenticated при отсутствии токена возвращает AuthInterceptor; KBService — проектируемый контракт.

2.4. Бизнес-правила и ограничения доступа

Функционирование компонента подчинено строгим логическим условиям и правилам разграничения ролей:

Правила SLA (Service Level Agreement): Каждому тикету при создании, в зависимости от выбранного приоритета (TicketPriority), рассчитывается точное предельное время ответа (slaDeadline). Для приоритета Urgent время ответа составляет 1 час, для High — 4 часа, для Medium — 24 часа, для Low — 72 часа. После наступления этой метки тикет помечается в интерфейсе как «Нарушен» и поднимается в самый верх очереди административной панели при сортировке «Критичные по SLA».

Жизненный цикл тикета: Обращение может переходить по состояниям: Open (Новый) -> InProgress (В работе) -> Resolved (Решен) -> Closed (Закрыт).

Оператор (Admin) имеет право перевести тикет в статус Closed на любом этапе (в интерфейсе — из статусов Open и InProgress), если проблема отпала.

Оператор переводит тикет в InProgress в момент нажатия кнопки «Взять».

Ограничения чата: Удаление или редактирование сообщений после отправки запрещено для обеспечения истории проведения аудита инцидентов.

Таблица 5 – Бизнес-правила и ограничения доступа

Правило	Описание
Доступ к экрану и API	/admin/support открывается только роли Admin (authGuard, roleGuard); все RPC SupportService требуют Bearer-токен, иначе Unauthenticated
Видимость тикетов	Admin видит все тикеты, остальные роли — только свои (authorId); GetTicket, ListMessages и AddMessage доступны автору или Admin, иначе PermissionDenied
Создание тикета	Любая роль создаёт тикет от своего имени; от имени другого пользователя (author_email) — только Admin. Новый тикет получает статус Open и срок SLA
Жизненный цикл и смена статуса	Статусы Open, InProgress, Resolved, Closed. Менять статус (UpdateTicketStatus) и закрывать тикет (CloseTicket) может только Admin; порядок переходов не проверяется, запрещено лишь менять закрытый тикет. InProgress назначает Admin исполнителем; Resolved и Closed записывают resolvedAt
SLA	slaDeadline = время создания + 1 ч (Urgent), 4 ч (High), 24 ч (Medium), 72 ч (Low). Статус SLA вычисляет интерфейс: «Нарушен» — срок прошёл, «Скоро нарушится» — осталось меньше 2 ч, иначе «В норме»
Сообщения и роли сообщений	Писать (AddMessage) могут автор тикета и Admin: роль Support — у Admin, User — у остальных. Первое сообщение CreateTicket всегда User; EscalateToOperator берёт роли (User, Ai, Support) из присланной истории. Ответ Admin назначает его исполнителем и переводит Open/Resolved → InProgress. Методов правки и удаления сообщений нет
Закрытый тикет	В тикет со статусом Closed нельзя добавить сообщение и сменить ему статус — FailedPrecondition; в интерфейсе поле ответа скрыто
Подтверждение действий	«Решено» и «Заккрыть» требуют подтверждения (confirm); при закрытии запрашивается итоговое решение, необязательное
ИИ-ассистент и эскалация на оператора	AskSupportAi возвращает шаблонный ответ (заглушка GigaChat); EscalateToOperator создаёт тикет с приоритетом Medium и переносит непустые сообщения истории. В интерфейсе не используются

Правило	Описание
Ошибки для пользователя	Ошибки загрузки и создания тикета выводятся текстом над очередью; при смене статуса и отправке сообщения ошибки не обрабатываются
Логирование	Отдельного аудита действий с тикетами нет; запросы попадают в общий HTTP-лог API
База знаний и поиск (проектируется)	Статей и поиска в коде нет; /faq, /applicant/faq и /notary/faq — заглушки. Чтение статей — всем ролям, ведение — Admin

3. UI КОМПОНЕНТЫ И РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

3.1. Перечень экранов и состояний

Интерфейс оператора поддержки встроен в оболочку административной панели портала Notary. Предусмотрены следующие основные экраны:

1. Экран Базы знаний (/applicant/faq, проектируется): Стартовая страница поддержки для заявителя. Содержит крупную поисковую строку по центру и сетку категорий (например, «Оплата услуг», «Документы для оценки», «Технические сбои»).
2. Экран списка обращений (/applicant/support, проектируется): Таблица личных тикетов заявителя с отображением текущего статуса, даты обновления и темы.
3. Рабочее место оператора (/admin/support): Сводная таблица тикетов в системе. Поддерживает динамическое обновление данных без перезагрузки страницы (опрос сервера каждые 10 с).
4. Экран чата по тикету (проектируется; сейчас чат оператора — панель на /admin/support): Двухпанельный интерфейс: слева — метаданные тикета, данные контрагента и таймер SLA; справа — окно диалога с прокруткой и полем ввода.

Таблица 6 – Перечень экранов компонента

Экран	Маршрут	Роль	Основные элементы	Действия
Страница поддержки (lib-admin-support)	/admin/support	Admin	Счётчики «Открытых», «Нарушено SLA», «Решено сегодня»; фильтры статуса, SLA и приоритета; сортировка; «Загрузка тикетов...»; текст ошибки	Фильтровать и сортировать; «+ Создать тикет»; список обновляется каждые 10 с
Очередь тикетов (lib-ticket-queue)	/admin/support	Admin	Колонки: тема и email, дата создания, статус, SLA, приоритет, осталось времени, действия; пустой список — «Нет тикетов, соответствующих фильтрам»	Выбрать тикет; «Взять» у тикета Open → InProgress
Чат оператора (lib-operator-chat)	/admin/support	Admin	Автор: email, дата регистрации, метка роли (user/admin — вычисляется по email); история сообщений; поле ответа, скрытое у Closed; «Выберите тикет из списка для начала работы»	Отправить ответ; «Решено»; «Закрыть» с итоговым решением
Окно создания тикета (lib-create-ticket-modal)	/admin/support	Admin	Тема, email пользователя, роль пользователя, приоритет SLA, описание; «Отмена», «Создать тикет»	Создать тикет от имени пользователя (CreateTicket с author_email)
Чат поддержки (проектируется)	/applicant/support, /notary/support	Applicant, Notary	Заглушка placeholder: «Чат/тикеты», «Вложения»; у заявителя также «SLA-статусы»	—

Экран	Маршрут	Роль	Основные элементы	Действия
Справочник: база знаний, FAQ, поиск (проектируется)	/applicant/faq, /notary/faq	Applicant, Notary	Заглушка placeholder: «База знаний», «FAQ», «Поиск по статьям»	—

3.2. Формы, фильтры и валидация

Форма создания тикета содержит валидацию на стороне клиента (шаблонная форма Angular, ngModel). Поля «Тема тикета» и «Email пользователя» являются строго обязательными. Кнопка отправки формы остается заблокированной (disabled), пока все условия валидации не выполнены. На экране очереди оператора реализован блок фильтрации. Доступна сортировка по дате, критичности SLA и приоритету, а также фильтры по статусу, SLA и приоритету. Нарушенные по времени SLA тикеты подсвечиваются мягким красным фоном метки SLA.

Таблица 7 – Поля формы создания тикета и правила проверки

Поле	Тип	Обязательное	Проверка	Сообщение об ошибке
Тема тикета (subject)	input text	Да	UI: не пусто после trim, иначе кнопка «Создать тикет» неактивна. Сервер: пусто после trim → InvalidArgument; min_len = 1 из proto отдельно не проверяется (protovalidate не подключён)	— (кнопка неактивна)
Email пользователя (author_email)	input email	Да	UI: не пусто и формат /^[^s@]+@[^s@]+\.[^s@]+\$/ — иначе кнопка неактивна. Сервер: author_email принимается только от Admin; пользователь с таким email должен существовать	— в форме; ошибка сервера «[not_found] User with email ... not found» выводится над очередью
Роль пользователя	select: user, vip, admin	Нет	Нет; значение на сервер не передаётся	—
Приоритет SLA (priority)	select: critical, high, medium, low	Да, по умолчанию medium	Значение всегда выбрано; critical → Urgent. Сервер: не задан → Medium	—
Описание проблемы (text)	textarea	Нет	Нет; непустое после trim сохраняется первым сообщением с ролью User	—

3.3. Пользовательские сценарии

1. Поиск информации и создание тикета (Заявитель; проектируется):

- Заявитель пытается решить возникшую проблему самостоятельно, используя строку полнотекстового поиска на экране FAQ. Клиентское приложение отправляет RPC-запрос SearchArticles (проектируется).

- Если среди возвращенных записей СУБД подходящего решения не нашлось, пользователь открывает форму обратной связи, заполняет обязательные поля и выбирает критичность. Сервис `SupportRpcService` (метод `CreateTicket`) регистрирует в базе данных сущность `Ticket` со статусом `Open` и автоматически рассчитывает метку времени `slaDeadline` (лимит времени на реакцию персонала). Интерфейс выполняет роутинг в созданную карточку чата (проектируется).
2. Обработка тикета из очереди (Оператор поддержки):
- Оператор службы поддержки обновляет экран общей очереди задач (`/admin/support`). Компонент запрашивает список инцидентов через метод `ListTickets`. СУБД возвращает записи, отсортированные по дате создания (сортировка по приоритету и SLA — в интерфейсе).
 - Оператор кликает по карточке инцидента и берет его на исполнение кнопкой «Взять» (`UpdateTicketStatus`). На уровне базы данных фиксируется идентификатор исполнителя (`assigneeId`), а статус тикета переходит в значение `InProgress`.
3. Ведение диалога в чате (Оператор и Заявитель; чат заявителя — проектируется):
- Оператор пишет текстовую рекомендацию по устранению сбоя и отправляет её в чат. Вызывается RPC-метод `AddMessage`, создающий запись в таблице `ticket_messages`.
 - В этот момент фоновый модуль бэкенда `NotificationService` генерирует моментальное push-уведомление для заявителя (проектируется). Заявитель заходит в личный кабинет, видит сообщение от поддержки, выполняет указанные действия и отправляет подтверждающий ответ обратно в чат (экран заявителя — проектируется).
4. Закрытие тикета (Оператор поддержки):
- Убедившись, что технический сбой устранен, оператор нажимает на интерфейсную кнопку «Закрыть» и подтверждает действие. Компонент чата отправляет финальную команду `CloseTicket` с итоговым решением.
 - В базе данных статус сущности `Ticket` окончательно меняется на `Closed`. На клиенте поле ввода ответа скрывается, а вся история переписки фиксируется в неизменяемом виде для последующего контроля качества работы службы поддержки.

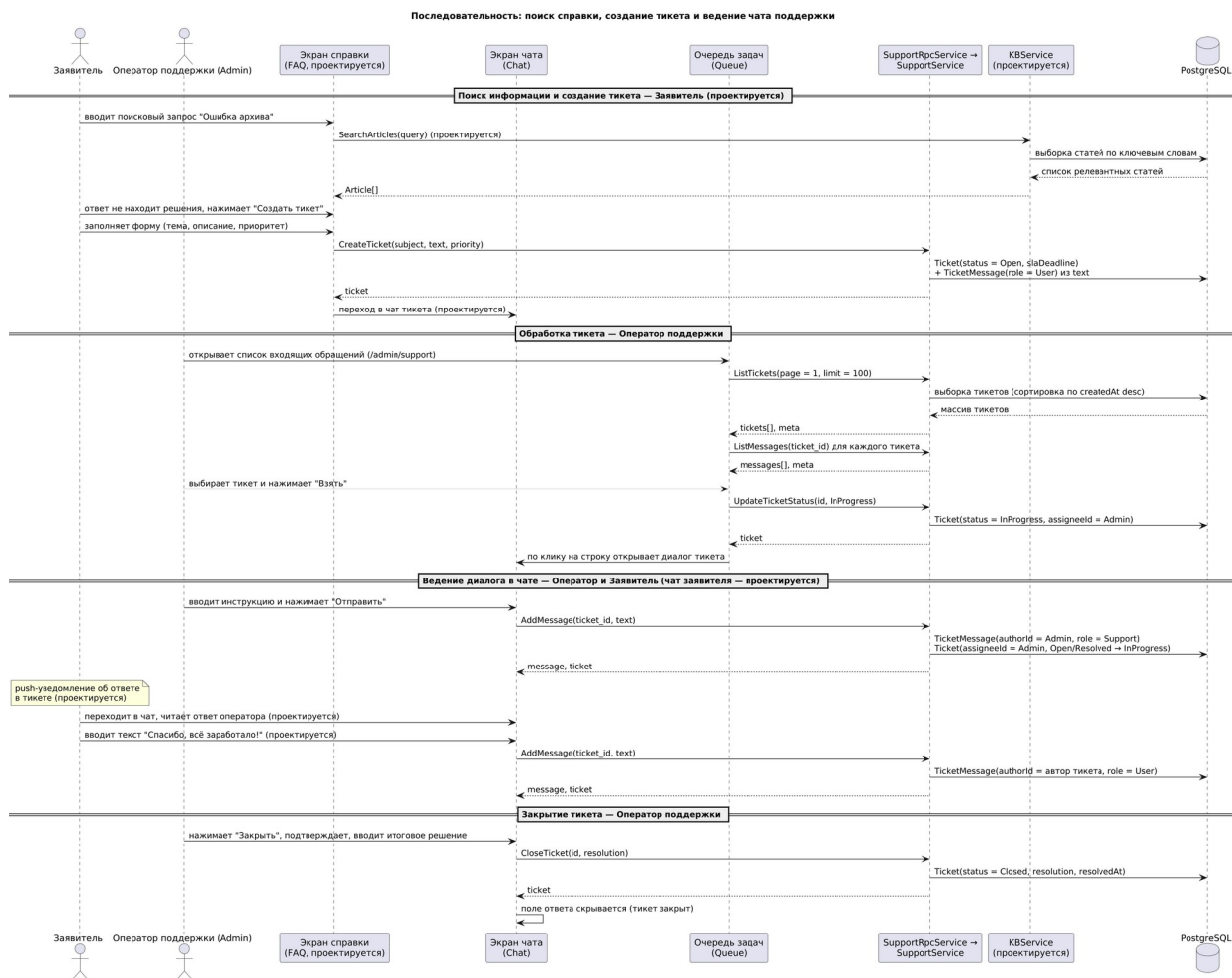


Рисунок 4 – Диаграмма сценариев

3.4. Скриншоты компонента

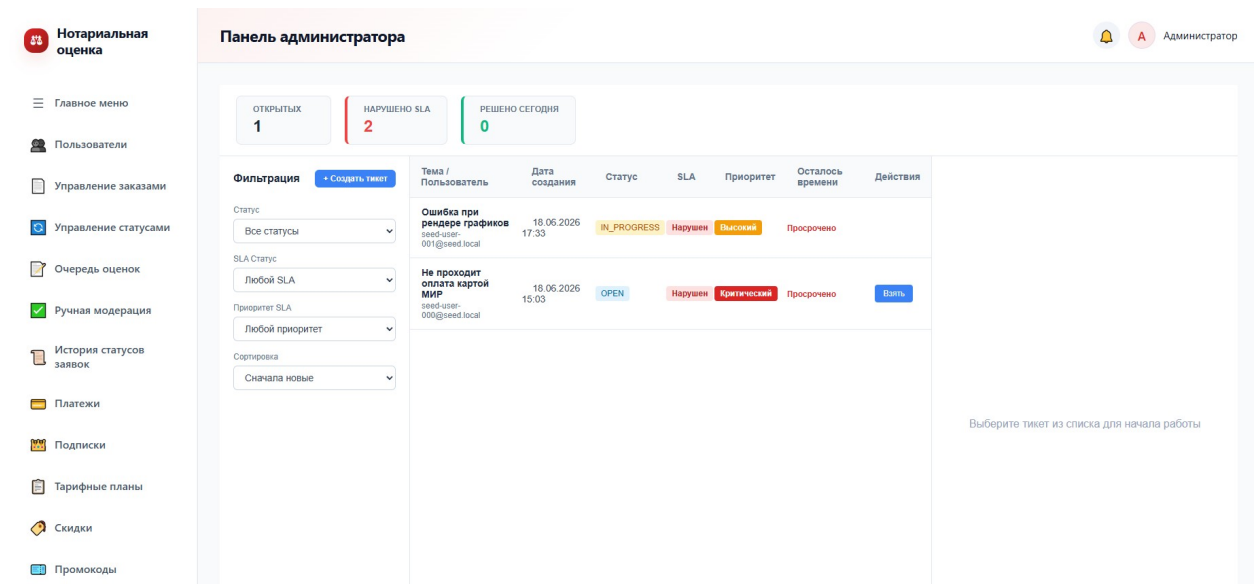


Рисунок 5 – Основная панель

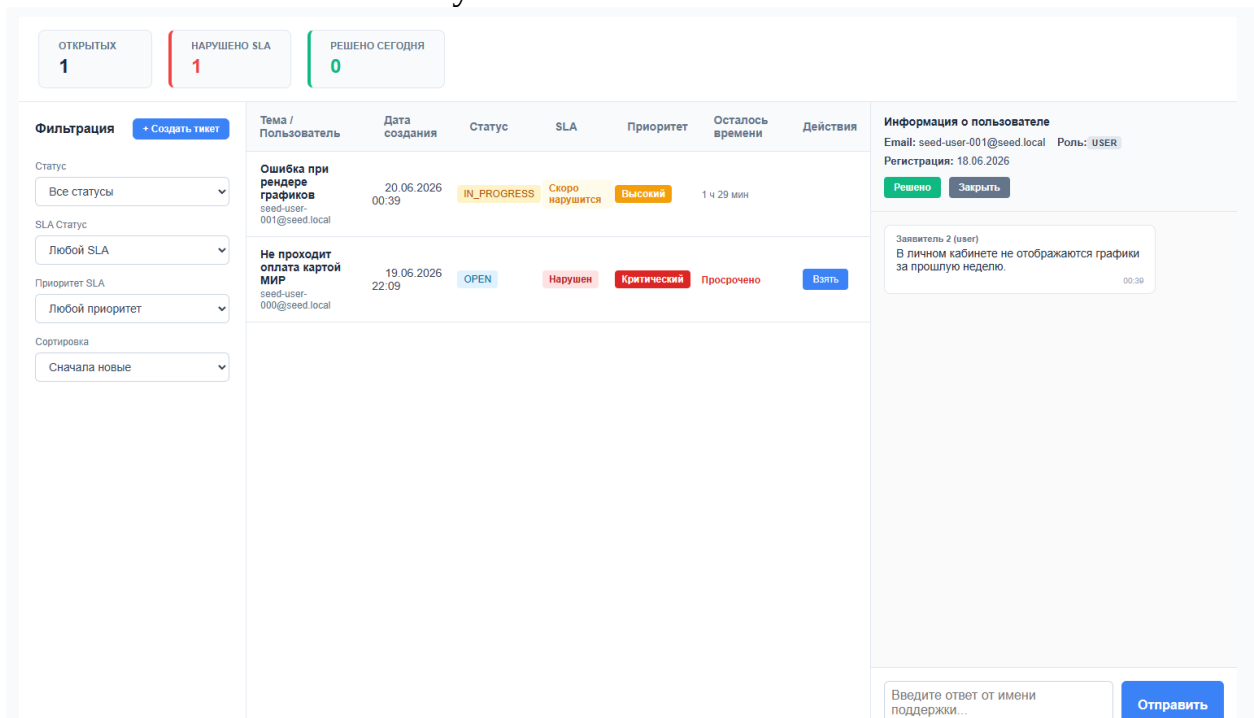


Рисунок 6 – Основная панель с окном чата

Создание нового тикета

×

Тема тикета *

Кратко опишите проблему

Email пользователя *

user@example.com

Роль пользователя

Обычный пользователь

▼

Приоритет SLA *

Средний (ответ в течение 24 часов)

▼

Описание проблемы

Подробно опишите проблему...

Отмена

Создать тикет

Рисунок 7 – Окно создания тикета

Заключение

В ходе выполнения курсовой работы был полностью спроектирован, структурирован и детально описан изолированный функциональный компонент «Чат поддержки, база знаний и поиск» для веб-портала автоматизации нотариальной деятельности Notary.

В рамках архитектурного проектирования были успешно решены все поставленные задачи:

1. Детализирована модель развертывания программного комплекса в контейнеризированной среде Linux/Docker с распределением портов, проксированием трафика через nginx и изоляцией ролей.
2. Создана нормализованная реляционная база данных и построена комплексная диаграмма классов UML, объединившая сущности тикетов, сообщений чата, структурированных статей FAQ (проектируется) и системных пуш-нотификаций (для тикетов — проектируется).
3. Разработана диаграмма компонентов с разделением общей архитектурной зоны ответственности на две независимые подсистемы (чат коммуникации и поисковый модуль справки (проектируется)), взаимодействующие по протоколу Connect RPC через четко документированные контракты.
4. Сформулированы бизнес-правила автоматического расчёта срока ответа (SLA) по приоритету и смены статусов обращений.
5. Подготовлено развернутое руководство пользователя с описанием трех базовых эксплуатационных сценариев для заявителей и операторов, а также спроектирована клиентская валидация экранных форм.

Разработанный компонент позволяет исключить использование сторонних систем связи (интерфейс пользователей — проектируется), повышает скорость обработки технических инцидентов, обеспечивает контроль сроков обработки обращений (SLA) и предоставляет пользователям удобный механизм самопомощи через базу знаний (проектируется).

Список используемых источников

1. Архитектурный шаблон и структура курсового задания проекта Notary: docs/coursework/structure.md.
2. Матрица распределения зон ответственности участников: docs/Responsobility.md.
3. Документация NestJS — Проектирование микросервисов и RPC-сервисов: [Электронный ресурс]. URL: <https://docs.nestjs.com/>
4. Руководство по разработке интерфейсов Angular (Template-driven Forms & Routing): [Электронный ресурс]. URL: <https://angular.dev/>
5. Спецификация протокола Connect RPC для TypeScript и Node.js: [Электронный ресурс]. URL: <https://connectrpc.com/>
6. Контракт сервиса поддержки SupportService: libs/shared/api-contracts/proto/notary/support/v1alpha1/support.proto.
7. Серверная логика поддержки (права доступа, SLA, статусы тикетов): libs/api/support/src/lib/support.service.ts.
8. Модели данных Ticket, TicketMessage, User: libs/api/shared/prisma/schema.prisma.
9. Интерфейс оператора поддержки: libs/web/admin/src/lib/features/support/.
10. Развертывание: docker-compose.yaml, apps/web/docker-compose.portal.yml, apps/web/Dockerfile, apps/web/nginx/portal.conf.

Приложение а. plantuml-код диаграмм

В данном приложении содержится исходный PlantUML-код для генерации всех графических схем и диаграмм, включенных в курсовую работу.

Исходный код диаграммы развертывания (Рисунок 1)

```
@startuml
```

```
title Диаграмма развертывания системы: Модуль поддержки и справки
```

```
skinparam defaultFontName DejaVu Sans
```

```
skinparam dpi 150
```

```
skinparam shadowing false
```

```
actor "Заявитель" as appl
```

```
actor "Оператор поддержки (Admin)" as support
```

```
node "Пользовательский браузер" as browser {
```

```
    artifact "Angular SPA\n/admin/support — модуль  
поддержки\n(libs/web/admin/.../features/support)\n/applicant/support  
(проектируется)\n/notary/support (проектируется)" as spa  
}
```

```
node "Виртуальный сервер (Linux, Docker)" as server {
```

```
    component "nginx portal (порт 80)\nстатика + проксирование\nConnect RPC  
(/notary.*) и REST (/api/*)" as nginx
```

```
    component "NestJS API (порт 3000)\nSupportRpcService -> SupportService\  
nKnowledgeBaseService (проектируется)\nNotificationService: оповещения\nпо тикетам (проектируется)" as api
```

```
    database "PostgreSQL (Prisma)\ntickets, ticket_messages, users\nArticle  
(проектируется)\nnotifications: для тикетов (проектируется)" as pg  
}
```

```
appl ..> spa : экраны поддержки\n(проектируется)
```

```
support --> spa
```

```
spa --> nginx : HTTP(S) через\nNginx Proxy Manager → :80
```

```
nginx --> api : Connect RPC / REST
```

```
api --> pg : Prisma
```

```
@enduml
```

Исходный код диаграммы классов (Рисунок 2)

```
@startuml
```

```
title Диаграмма классов: чат поддержки, база знаний и поиск
```

```
skinparam defaultFontName DejaVu Sans
```

```
skinparam dpi 150
```

```
skinparam shadowing false
```

```
skinparam classAttributeIconSize 0
```

```
enum TicketStatus {  
    Open  
    InProgress  
    Resolved  
    Closed  
}
```

```
enum TicketPriority {  
    Low  
    Medium  
    High  
    Urgent  
}
```

```
enum Role {  
    Applicant  
    Notary  
    Admin  
}
```

```
enum TicketMessageRole {  
    User  
    Ai  
    Support  
}
```

```
class User {  
    id : UUID «PK»  
    email : varchar «unique, not null»  
    passwordHash : varchar?  
    fullName : varchar «not null»  
    role : Role «not null»  
    phoneNumber : varchar?  
    isActive : boolean «not null»  
}
```

```
class Ticket {  
    id : UUID «PK»  
    subject : varchar «not null»  
    status : TicketStatus «not null»  
    priority : TicketPriority «not null»  
    authorId : UUID «FK, not null»  
    assigneeId : UUID? «FK»  
    slaDeadline : timestamp «not null»  
}
```

```

    resolution : text?
    resolvedAt : timestamp?
    createdAt : timestamp «not null»
    updatedAt : timestamp «not null»
}

```

```

class TicketMessage {
    id : UUID «PK»
    ticketId : UUID «FK, not null»
    authorId : UUID «FK, not null»
    role : TicketMessageRole «not null»
    text : text «not null»
    attachmentIds : text[]
    createdAt : timestamp «not null»
}

```

```

class Article <<проектируется>> #line.dashed {
    id : UUID «PK»
    title : varchar «not null»
    content : text «not null»
    category : varchar «not null»
    authorId : UUID «FK, not null»
    views : int «not null»
    createdAt : timestamp «not null»
    updatedAt : timestamp «not null»
}

```

```

class Notification {
    id : UUID «PK»
    userId : UUID «FK, not null»
    type : NotificationType «not null»
    message : text «not null»
    sentAt : timestamp «not null»
    readAt : timestamp?
}

```

```

User "1" --> "0..*" Ticket : "создаёт (authorId)"
User "0..1" --> "0..*" Ticket : "обрабатывает (assigneeId)"
User "1" --> "0..*" TicketMessage : "отправляет (authorId)"
User "1" ..> "0..*" Article : "пишет (authorId)\nпроектируется"
User "1" --> "0..*" Notification : "получает"

```

```

Ticket "1" --> "0..*" TicketMessage : "содержит сообщения"
Ticket ..> Notification : "триггерит оповещения\n(проектируется)"

```

```
TicketStatus -[hidden]down- TicketPriority
TicketPriority -[hidden]down- Role
Role -[hidden]down- TicketMessageRole
@enduml
```

Исходный код диаграммы компонентов (Рисунок 3)

```
@startuml
title Диаграмма компонентов: Чат поддержки и База знаний
skinparam defaultFontName DejaVu Sans
skinparam dpi 150
skinparam shadowing false

package "Frontend (Angular SPA)" {
    component "TicketQueueComponent" as queue
    component "OperatorChatComponent" as chatUI
    component "ArticleSearchComponent\n(проектируется)" as searchUI
    #line.dashed
    component "ArticleViewComponent\n(проектируется)" as viewUI #line.dashed
    interface "SupportService (Connect Client)\nSupportApiService" as ticketClient
    interface "KBService (Connect Client)\n(проектируется)" as kbClient
}

package "Backend (NestJS API)" {
    component "SupportRpcService" as ticketRpc
    component "KnowledgeBaseRpcService\n(проектируется)" as kbRpc
    #line.dashed
    component "SupportService" as ticketCore
    component "KnowledgeBaseService\n(проектируется)" as kbCore #line.dashed
    component "NotificationService" as notifyCore
}

database "PostgreSQL" as db

queue --> ticketClient
chatUI --> ticketClient
searchUI ..> kbClient
viewUI ..> kbClient

ticketClient --> ticketRpc : RPC
kbClient ..> kbRpc : RPC

ticketRpc --> ticketCore
kbRpc ..> kbCore
```

```
ticketCore --> db : Prisma
kbCore ..> db : Prisma
ticketCore ..> notifyCore : оповещения (проектируется)
@enduml
```

Исходный код диаграммы сценариев (Рисунок 4)

```
@startuml
title Последовательность: поиск справки, создание тикета и ведение чата
поддержки
skinparam defaultFontName DejaVu Sans
skinparam dpi 150
skinparam shadowing false
```

```
actor "Заявитель" as A
actor "Оператор поддержки (Admin)" as O
participant "Экран справки\n(FAQ, проектируется)" as FAQ
participant "Экран чата\n(Chat)" as CHAT
participant "Очередь задач\n(Queue)" as Q
participant "SupportRpcService → \nSupportService" as S
participant "KBService\n(проектируется)" as KB
database "PostgreSQL" as DB
```

```
== Поиск информации и создание тикета — Заявитель (проектируется) ==
A -> FAQ : вводит поисковый запрос "Ошибка архива"
FAQ -> KB : SearchArticles(query) (проектируется)
KB -> DB : выборка статей по ключевым словам
DB --> KB : список релевантных статей
KB --> FAQ : Article[]
A -> FAQ : ответ не находит решения, нажимает "Создать тикет"
A -> FAQ : заполняет форму (тема, описание, приоритет)
FAQ -> S : CreateTicket(subject, text, priority)
S -> DB : Ticket(status = Open, slaDeadline)\n+ TicketMessage(role = User) из
text
S --> FAQ : ticket
FAQ -> CHAT : переход в чат тикета (проектируется)
```

```
== Обработка тикета — Оператор поддержки ==
O -> Q : открывает список входящих обращений (/admin/support)
Q -> S : ListTickets(page = 1, limit = 100)
S -> DB : выборка тикетов (сортировка по createdAt desc)
DB --> S : массив тикетов
S --> Q : tickets[], meta
Q -> S : ListMessages(ticket_id) для каждого тикета
S --> Q : messages[], meta
```

O -> Q : выбирает тикет и нажимает "Взять"
Q -> S : UpdateTicketStatus(id, InProgress)
S -> DB : Ticket(status = InProgress, assigneeId = Admin)
S --> Q : ticket
Q -> CHAT : по клику на строку открывает диалог тикета

== Ведение диалога в чате — Оператор и Заявитель (чат заявителя — проектируется) ==

O -> CHAT : вводит инструкцию и нажимает "Отправить"
CHAT -> S : AddMessage(ticket_id, text)
S -> DB : TicketMessage(authorId = Admin, role = Support)\nTicket(assigneeId = Admin, Open/Resolved → InProgress)
S --> CHAT : message, ticket
Note over A : push-уведомление об ответе\пв тикете (проектируется)
A -> CHAT : переходит в чат, читает ответ оператора (проектируется)
A -> CHAT : вводит текст "Спасибо, всё заработало!" (проектируется)
CHAT -> S : AddMessage(ticket_id, text)
S -> DB : TicketMessage(authorId = автор тикета, role = User)
S --> CHAT : message, ticket

== Закрытие тикета — Оператор поддержки ==

O -> CHAT : нажимает "Закрыть", подтверждает, вводит итоговое решение
CHAT -> S : CloseTicket(id, resolution)
S -> DB : Ticket(status = Closed, resolution, resolvedAt)
S --> CHAT : ticket
CHAT -> CHAT : поле ответа скрывается (тикет закрыт)
@enduml